

CONTRACT DOCUMENT DELIVERABLE

IDM UID

DN2BMQ

VERSION CREATED ON / VERSION / STATUS

10 Feb 2026 / 1.0 / Signed

EXTERNAL REFERENCE

ROX and Research and Development Report **D5.7 Software Test Plan**

Software Test Plan for firmware and software of AWG, and simple user manual

Acceptance Process			
	Name	Action	Job Title / Affiliation
Author	Makowski D.	10 Feb 2026:signed	COSYLAB d.d. (EU)
Co-Authors	Mielczarek A.	10 Feb 2026:signed	COSYLAB d.d. (EU)
Reviewers	Esquembri S.		Control Systems integration Enginee...
Accepted By	Leveque A.		Diagnostics I&C Coordinator
Information Protection Level: Non-Public - Unclassified			
RO: Leveque Arthur			
Read Access	LG: DICAD2, LG: TED Management , AD: ITER, AD: External Collaborators, AD: OBS - Diagnostics Program (DIAG) - EXT, AD: IO_Director-General, AD: External Management Advisory Board, AD: OBS - Central Integration Division (CID), AD: IDM_Controller, AD: OBS - Central Integration Division (CID) - EXT, AD...		

<i>Change Log</i>			
D5.7 Software Test Plan (DN2BMQ)			
<i>Version</i>	<i>Latest Status</i>	<i>Issue Date</i>	<i>Description of Change</i>
v0.0	In Work	16 Jan 2026	
v1.0	Signed	10 Feb 2026	Software test plan for arbitrary waveform generator (uTCA function generator) - high speed version (2 channels, 1 GSps).

SOFTWARE TEST PLAN

MFMC+ Arbitrary Waveform Generator

[Abstract](#)

This document defines a Software Test Plan for the MFMC+ Arbitrary Waveform Generator project. It might be used as standalone solution or as a base for further MicroTCA.4 developments. The design runs on an FMC carrier board with Kintex Ultrascale+ FPGA, accompanied by the dmtcauni driver and a set of Python scripts.

Table of contents

1	INTRODUCTION	2
1.1	Purpose and Scope	2
1.2	Overview	2
1.3	References	3
1.4	Definitions	3
2	DETAILS OF THE TESTING PROCESS	4
2.1	Definition of test levels.....	4
2.2	Test administration	4
2.2.1	Test reporting requirements	4
2.2.2	Test deliverables	4
2.2.3	Estimates, cost, and training	4
2.3	Test Environment.....	4
2.3.1	MicroTCA.4 Arbitrary Waveform Generator Hardware Installation	4
2.4	Features traceability	6
2.4.1	MTCA-AWG Firmware Functions	6
3	MTCA-AWG SOFTWARE TEST PLAN.....	7
3.1	Scope.....	7
3.1.1	Test items and their identifiers	7
3.2	Approach	7
3.2.1	Testing Methods	7
3.2.2	Item pass/fail criteria	7
3.3	Test Environment.....	7
3.4	Test Procedures.....	8
3.4.1	Hardware Installation Tests.....	8
3.4.2	Software Installation Tests	9
3.4.3	Functional Tests	16
3.5	MicroTCA.4 FPGA Reference System STR Template	24
3.5.1	Summary	24
3.5.2	Hardware Installation Tests.....	24
3.5.3	Software Installation Tests	24
3.5.4	Bitstream Synthesis and Upload Tests	25
3.5.5	Functional Tests	25

1 INTRODUCTION

1.1 Purpose and Scope

This document is a test plan for the MicroTCA.4 Arbitrary Waveform Generator. It describes the test procedures used to verify that the design meets its established requirements.

The tests should be executed by CODAC Core System developers before any release.

In this document CODAC Core System version 7.3.0 is assumed. See [RD1] to install CODAC Core System. Installed system version can be obtained and checked by executing command `codac-version`.

The main goal of performing the test is to evaluate operation of the design as a source for test pulses emulating operation of a PMT with an amplifier, e.g for the 55.GE (HXRM) system.

1.2 Overview

Software support of the MTCA-AWG project includes Linux Device Driver together with Python wrappers implementing console application and a simple GUI control panel.

1.3 References

Ref#	Document Name	IDM Ref	Version
[RD1]	CODAC Core System Installation Manual	ITER_D_33JNKW	7.3

1.4 Definitions

AMC	Advanced Mezzanine Card
API	Application Programming Interface
AWG	Arbitrary Waveform Generator
CCS	CODAC Core System
CLI	Command Line Interface
CODAC	Control, Data Access, and Communications
DMA	Direct Memory Access
EPICS	Experimental Physics and Industrial Control System
FMC	FPGA Mezzanine Card
FPGA	Field-Programmable Gate Array
HXRM	Hard X-Ray Monitor
MCH	MicroTCA Carrier Hub
MicroTCA	Micro Telecommunications Computing Architecture
MMC	Module Management Controller
MTCA	See MicroTCA
PCIe	Peripheral Component Interconnect Express
PLL	Phase-Locked Loop
PMT	PhotoMultiplier Tube
PROM	Programmable Read-Only Memory
STP	Software Test Plan

2 DETAILS OF THE TESTING PROCESS

2.1 Definition of test levels

Following test levels are defined in this test plan to organize the testing activity.

MicroTCA.4 FPGA Reference System	MTCA-AWG
This level includes test procedures for the MicroTCA.4 Arbitrary Waveform Generator. The tests evaluate the Linux Device Driver, Python wrappers, FPGA firmware and the hardware.	

2.2 Test administration

2.2.1 Test reporting requirements

Test reports shall be submitted in ITER Document Management System (IDM) using the Software Test Report Template. In case of errors or anomalies a test log shall be also included in the report document.

2.2.2 Test deliverables

Following documents shall be delivered in IDM.

- MTCA-AWG – Software Test Report

2.2.3 Estimates, cost, and training

The members of testing team should have basic knowledge of Linux operating system and should have general knowledge on analogue data acquisition systems as well as be familiar with basic operation of MicroTCA.4-based systems.

Estimated time required to perform all tasks specified in this plan is 2 hours.

2.3 Test Environment

2.3.1 MicroTCA.4 Arbitrary Waveform Generator Hardware Installation

The MTCA-AWG project was designed to run on the AIES MFMC+AFG4-1 module (arbitrary waveform generator with Ultra Scale+ FPGA for MTCA.4 Systems, 2 channels, 1 GSps). The module shall be installed in an MicroTCA.4 chassis. The chassis shall be operational, i.e. include a working power supply, cooling unit and MCH. The test system shall include also a CPU running CCS connected to the chassis with a PCI Express interface. The CPU can be either internal or external to the crate.

In order to synthesize the FPGA project a machine with Vivado Design Suite 2023.2 is required.

The test plan assumes use of two test setups. The first one requires an oscilloscope for observation of the generated waveforms, as is illustrated in Figure 1.

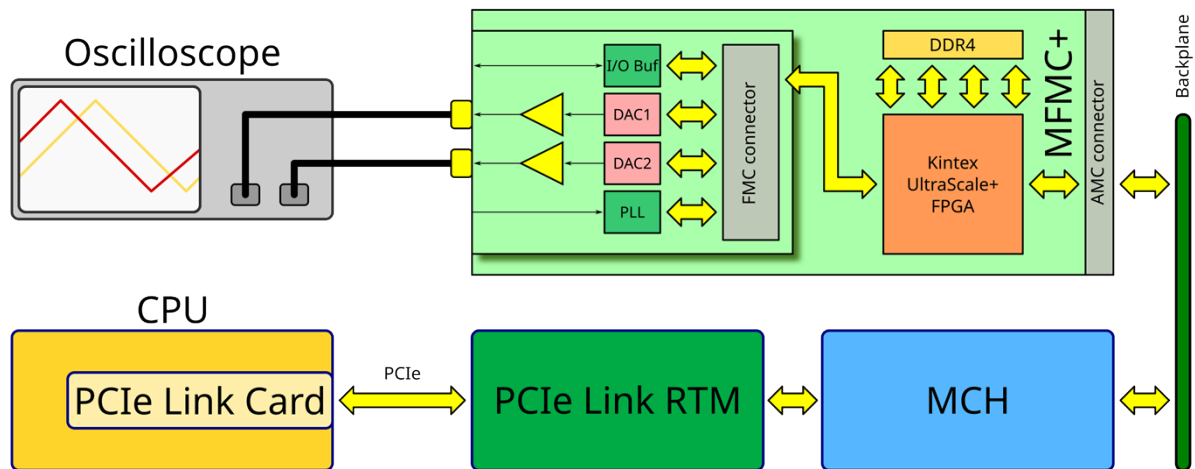


Figure 1. MTCA-AWG test hardware

The second, optional, test setup facilitates automated waveform analysis with help of the ADQ14 data acquisition module. During the test the AWG emulates an active PMT. For the test the first output of the DAC module shall be connected with an SMA cable through a 20 dB attenuator to one of the ADQ14 inputs, as illustrated in Figure 2.

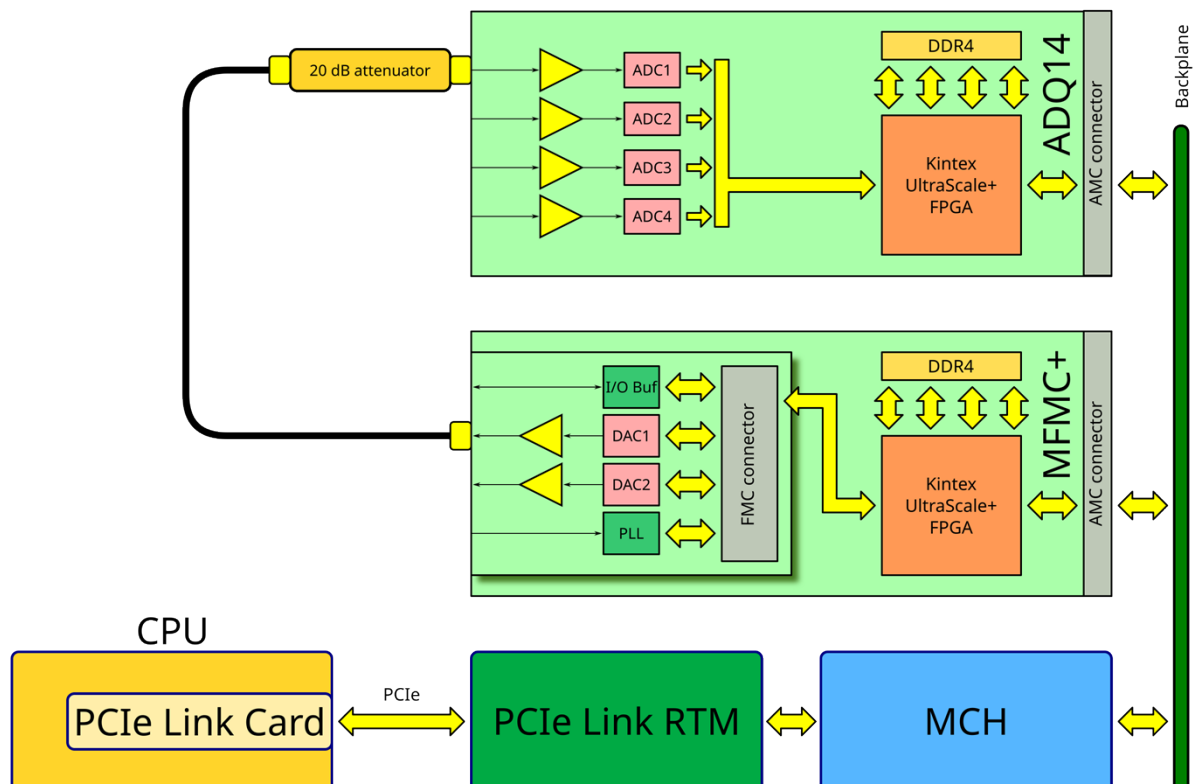


Figure 2. MTCA-AWG test hardware

2.4 Features traceability

2.4.1 MTCA-AWG Firmware Functions

Requirement ID	Description	Test type	Test ID list
MTCA-AWG-F-1	Complete source code available with guidelines on how to use and synthesize it.	C	TP-MTCA-AWG-B-0001
MTCA-AWG-F-2	Firmware upgrade over PCIe	C	TP-MTCA-AWG-B-0002
MTCA-AWG-F-3	Generate a set of predefined waveforms with variable amplitude, offset and frequency.	C	TP-MTCA-AWG-F-0002
MTCA-AWG-F-4	Load and reproduce an arbitrary waveform	C	TP-MTCA-AWG-F-0003, TP-MTCA-AWG-F-0004, TP-MTCA-AWG-F-0005

3 MTCA-AWG SOFTWARE TEST PLAN

3.1 Scope

3.1.1 Test items and their identifiers

The items being tested by this STP are:

Test Item	Version	Description
m-mtca-awg-2c	v1.0	Provides an FPGA source code of MTCA-AWG
m-kmod-dmtcauni	v3.5.0	The universal driver for TUL-DMCS MTCA modules
m-mtca-awg-python	v1.0	Python scripts supporting the AWG

3.2 Approach

3.2.1 Testing Methods

All tests presented in this STP are prepared in accordance with black box testing methodology. Therefore, the tests are focused on the functionality of an application as opposed to its internal structure. The tests procedures describe what stimulus shall be provided to the tested block and how it shall respond.

3.2.2 Item pass/fail criteria

Overall criteria for each test item to pass or fail:

Test Item	Criteria
m-mtca-awg-2c, m-kmod-dmtcauni, m-mtca-awg-python	Test item is passed if all pass criteria are met. Otherwise, the test item is considered failed. Intermediate results are not considered.

3.3 Test Environment

Following hardware components are required to perform the tests:

- MTCA.4 chassis with: power supply, cooling unit, and MCH
- A CPU running CCS 7.3 – internal, or external with PCIe link (min. x4, gen.2)
- MPMC+AFG4-1 AWG module, 2 channels, 1 GSps
- An oscilloscope with bandwidth not less than 100 MHz
- ADQ14 module in the MicroTCA form factor
- SMA cable
- 20 dB SMA attenuator
- Machine with Vivado Design Suite 2023.2 – for FPGA project synthesis

3.4 Test Procedures

3.4.1 Hardware Installation Tests

TP-MTCA-AWG-H-0001	Power on MicroTCA.4 chassis and check PCIe operation
Description	Turn on the MTCA.4 chassis and make sure that the MFMC+AFG4-1 and ADQ14 devices are visible to the CPU.
Pre-requisite	MTCA.4 chassis equipped with: • Power supply • Cooling unit • MCH • CPU or PCIe link to the external CPU • MFMC+AFG4-1 AWG module, 2 channels, 1 GSps, called further MTCA-AWG • ADQ14 module
Procedure	1. Make sure that all the required modules are present and their AMC handles are closed. 2. Turn on the power supply of the MTCA.4 chassis. 3. Wait until all the modules listed in pre-requisites are activated. Their blue LEDs shall finally turn on. The green LEDs of MFMC+ shall be ON. 4. If an external CPU is used, start it now. Wait for the CCS to start. 5. Log in to the CPU and execute the following commands: <pre>\$ lspci -d 1b37: 10:00.0 Signal processing controller: Signal Processing Devices Sweden AB ADQ14 \$ lspci -d 9f83: 0f:00.0 Serial controller: Device 9f83:0dac</pre> 6. Verify that both devices were found.
Pass Criteria	1. All required MTCA.4 modules are activated, blue LEDs are OFF. 2. The MTCA-AWG and ADQ14 devices are visible in the PCIe bus.

3.4.2 Software Installation Tests

TP-MTCA-AWG-S-0001	Download and install the dmtcauni driver
Description	Checkout the dmtcauni driver source code from GIT, build and install RPMs.
Pre-requisite	TP-MTCA-AWG-H-0001, driver not installed (execute TP-MTCA-AWG-S-0002 if uninstall is required)
Procedure	1. Log in to the CPU. 2. Install the build pre-requisites: <pre>\$ sudo yum install cmake codac-core-7.3-cpp-devel elfutils-libelf-devel</pre> 3. Download the dmtcauni device driver from GIT: <pre>\$ cd /tmp \$ git clone ssh://git@git.iter.org/dic/m-kmod-dmtcauni.git --branch v3.5.0</pre> 4. Package and install the driver: <pre>\$ cd /tmp/m-kmod-dmtcauni \$ mvn clean package \$ sudo ./rpm_install.sh</pre> 5. The installation procedure will load the kernel module automatically. Check if both expected driver nodes were created: <pre>\$ ls /dev/gen_* /dev/gen_12.ctrl /dev/gen_12.dma</pre>
Pass Criteria	1. Source code successfully downloaded from GIT. 2. No errors during compilation and installation. 3. Two “gen” nodes visible under the /dev/ directory corresponding to the MicroTCA slot in which the tested module was installed.

The following test removes the software support packages. In most cases these shall be executed after the rest of the STP is completed or as a pre-requisite for clean installation.

TP-MTCA-AWG-S-0002	dmtcauni driver uninstall procedure
Description	Remove dmtcauni driver from the CPU.
Pre-requisite	TP-MTCA-AWG-S-0001
Procedure	1. Log in to the CPU. 2. Use the following command to check which RPMs related to the driver are installed: <pre>\$ rpm -qa "codac-core-`codac-version -v`-kmod-dmtcauni*" codac-core-7.3-kmod-dmtcauni-tools-7.3.0.v3.5.0-0.el8.x86_64 codac-core-7.3-kmod-dmtcauni-lib-devel-7.3.0.v3.5.0-0.el8.x86_64 codac-core-7.3-kmod-dmtcauni-4.18.0-348.7.1.el8_5-7.3.0.v3.5.0-0.el8.x86_64 codac-core-7.3-kmod-dmtcauni-lib-7.3.0.v3.5.0-0.el8.x86_64</pre> 3. Unload Linux kernel module: <pre>\$ sudo rmmod dmtcauni</pre> 4. Uninstall all the driver components: <pre>\$ sudo rpm -e --nodeps \$(rpm -qa "codac-core-`codac-version -v`-kmod-dmtcauni*")</pre> In case this step fails please change directory to /tmp/ and try again. 5. Check that the RPMs are not installed using command from step 2. Empty output is expected. 6. Remove the downloaded source files (if any): <pre>\$ rm -rf /tmp/m-kmod-dmtcauni/</pre>
Pass Criteria	1. In point 3, the module gets unloaded successfully. 2. No RPMs are listed in step 5.

TP-MTCA-AWG-S-0003	Download and install the ADQ14 device drivers
Description	Get the ADQ14 SDK and install it.
Pre-requisite	spd-adq-pci-dkms, libadq0, adqtools not installed
Procedure	(Skip this test if not using ADQ14)
1. Log in to the CPU. 2. Download the ADQ14 SDK from this link. 3. Unpack the archive:	
<pre>\$ tar xvfz TSPD_SDK_linux_2024.1.tar.gz</pre>	
4. Package and install the driver:	
<pre>\$ cd TSPD_SDK_linux_2024.1/packages/centos_8_64-bit/ \$ sudo yum install ./spd-adq-pci-dkms-1.24-1.noarch.rpm \$ sudo yum install ./libadq0-2024.1-1.el8.x86_64.rpm \$ sudo yum install ./adqtools-2024.1-1.el8.x86_64.rpm</pre>	
5. Add device access permissions:	
<pre>\$ usermod -a -G adq codac-dev</pre>	
6. Reboot CPU and check if the ADQ14 is visible in the device tree:	
<pre>\$ ls /dev/adq* /dev/adq_pcie_16_0_0</pre>	
Pass Criteria	
1. Source code successfully downloaded from SVN. 2. No errors during compilation and installation. 3. An “adq” node visible under the /dev/ directory corresponding to the MicroTCA slot in which the tested module was installed.	

TP-MTCA-AWG-S-0004	Download and install the ADQscope
Description	Get the ADQScope binary from the SVN and run it
Pre-requisite	ADQScope not installed
Procedure	(Skip this test if not using ADQ14)

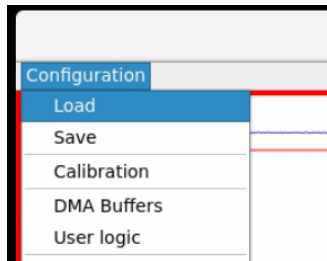
1. Log in to the CPU.
2. Download the ADQscope from [this link](#).
3. Rename it to *qtadqscope* and add an 'executable' flag to the binary:

```
$ mv qtadqscope-codac-7.3-sdk-2024 qtadqscope
$ chmod +x qtadqscope
```

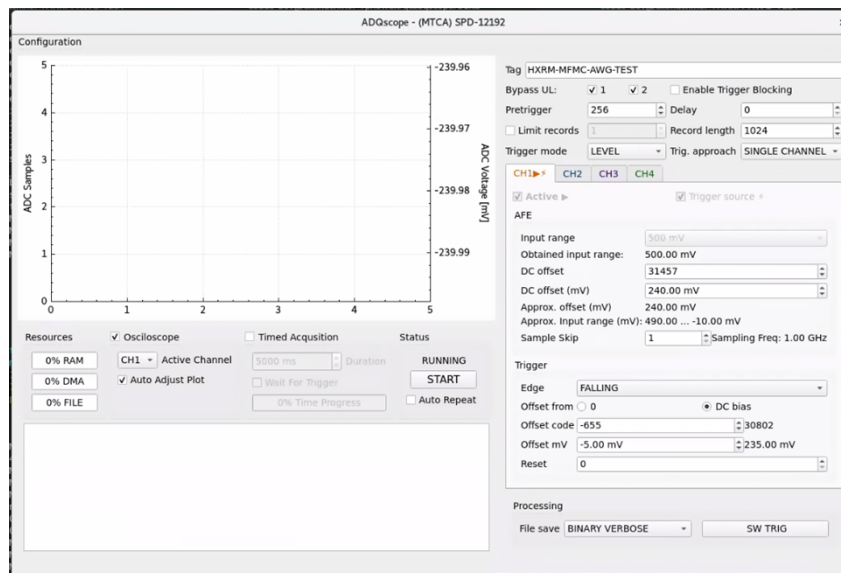
4. Download settings file from [this link](#).
5. Run the application:

```
$ ./qtadqscope
```

6. Import settings using Configuration → Load



7. Find “adq14_adqgen_settings.json” and load it
8. The state of the application should be as follows:



Pass Criteria

1. Source code successfully downloaded from SVN.
2. No errors during installation.
3. The ADQ14 was successfully detected by the ADQscope
4. Acquisition settings successfully loaded

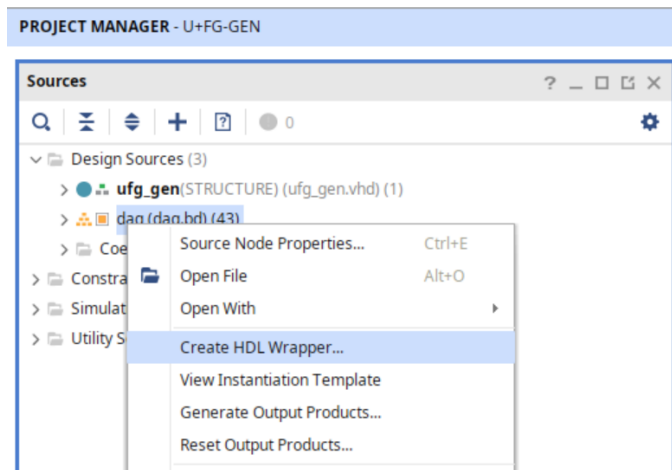
Bitstream Synthesis and Upload Tests (Optional)

TP-MTCA-AWG-B-0001	Download and build the FPGA project
Description	Checkout the MTCA-AWG source code from GIT and build it using Vivado.
Pre-requisite	Machine with Vivado Design Suite 2023.2
Procedure	

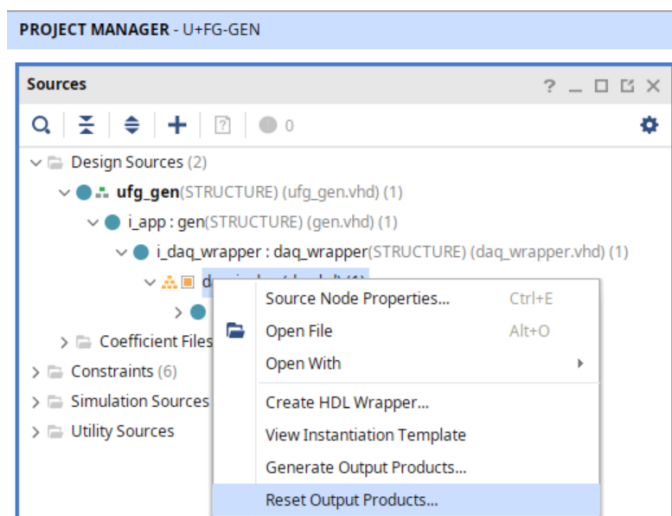
1. Log in to the FPGA development machine with Vivado Design Suite 2023.2
2. Download the MTCA-AWG source code from GIT:

```
$ git clone ssh://git@git.iter.org/dic/m-mtca-awg-2c.git --branch v1.0
$ cd m-mtca-awg-2c/
$ git submodule init
$ git submodule update
```

3. Open the U+FG-GEN.xpr project in Vivado
4. Right click on the block design and select “Create HDL Wrapper...”, then click “OK” in a pop-up window and wait for the “Sources” tree to update.



5. Find the block design in “Sources”, right click it and select “Reset Output Products...”:



6. Press the “Generate Bitstream” button from the “Flow Navigator”. The process shall take less than an hour to complete.

Pass Criteria	
1. Code downloaded without errors 2. Bitstream successfully generated	

TP-MTCA-AWG-B-0002	Program the FPGA PROM
Description	Use the firmware upgrade to load a new design to the FPGA configuration memory and boot from it.
Pre-requisite	TP-MTCA-AWG-S-0001, TP-MTCA-AWG-B-0001
Procedure	

1. Log in to the CPU.
2. Copy the m-mtca-awg-2c/U+FG-GEN.runs/impl_1/ufg_gen.bit from the FPGA development machine to the CPU used for tests.
3. Make sure that there are no other active DMCS modules in the crate but the MFMC+.

```
$ lspci -d 9f83:
0f:00.0 Serial controller: Device 9f83:0dac
```

Only one board shall be visible in output of the `lspci` command. If the STP has to be executed in crate with other active DMCS devices all the invocations of the `dmtcauni_fwup` tool will have to be extended with the `-d` argument and specification of a control node of a particular device to be upgraded (e.g. `dmtcauni_fwup -d /dev/gen_12.ctrl1`).

4. Obtain the current status of the firmware stored in the non-volatile memory:

```
$ dmtcauni_fwup

Accessing the SPI memory... success
Flash size is 256 Mbit
Manufacturer: Spansion
QUAD bit is SET

Reading first page... done
===== BITSTREAM IN SPI FLASH =====
Xilinx bitstream starting at 0xBC
  Bit name: ufg_gen
  UserID: 0xFFFFFFFF
  Version: 2023.2
  FPGA: xcku1p-fv1517-1-i
  Synth date: 2025/12/10
  Synth time: 13:11:58
=====
```

If the SPI memory was successfully accessed, the hardware is ready to be upgraded.

5. Erase and program the configuration memory using the .bit file:

```
$ dmtcauni_fwup -b mfmc_daq.bit -ep
```

The process will take several minutes.

In an unlikely event that the programming failed for any reason the firmware image stored in the non-volatile memory can be broken or incomplete. Avoid restarting the

AMC module and retry from step 3. You can also try rebooting the CPU, but NOT the crate nor the AMC module. If the board is lost (no green LED, no device visible in PCIe) the FPGA has to be programmed using Xilinx/AMD Platform Cable. Then, after rebooting the CPU, continue from step 3.

6. Only if the step 5 was finished successfully, restart the AMC module, e.g. by pulling the AMC handle for several seconds and then pushing it back again. The module does not have to be removed from crate. This operation will cause FPGA to be configured using the bitstream stored in the non-volatile memory.
7. Restart the CPU. This ensures, that the PCIe bus will be fully re-enumerated
8. Compare the local bitstream with the non-volatile memory contents:

```
$ dmtcauni_fwup -b mfmc_daq.bit

Examining bitstream file... ok
===== BITSTREAM IN BIT FILE =====
Xilinx bitstream starting at 0xBC
  Bit name: ufg_gen
    UserID: 0xFFFFFFFF
    Version: 2023.2
      FPGA: xcku1lp-ffve1517-1-i
  Synth date: 2025/12/10
  Synth time: 13:11:58
=====

Accessing the SPI memory... success
Flash size is 256 Mbit
Manufacturer: Spansion
QUAD bit is SET

Reading first page... done
===== BITSTREAM IN SPI FLASH =====
Xilinx bitstream starting at 0xBC
  Bit name: ufg_gen
    UserID: 0xFFFFFFFF
    Version: 2023.2
      FPGA: xcku1lp-ffve1517-1-i
  Synth date: 2025/12/10
  Synth time: 13:11:58
=====
```

This command will only read and print the bitstream headers from the configuration memory and from the referenced file. It will not perform any programming operation.

Pass Criteria
1. The two bitstream headers displayed in the final step shall provide the same information, confirming that the firmware image was updated and the board has successfully booted it.

3.4.3 Functional Tests

TP-MTCA-AWG-F-0001	Download and test the Python Support
Description	Checkout the Python sources from GIT and use them to communicate with the module.
Pre-requisite	TP-MTCA-AWG-S-0001
Procedure	1. Log in to the CPU. 2. Install the required packages: <pre>\$ sudo yum install python3-numpy python3-qt5 python3-pyqt5-sip</pre> 3. Download the Python Support from GIT: <pre>\$ git clone ssh://git@git.iter.org/dic/m-mtca-awg-python.git --branch v1.0</pre> 4. Enter the scripts directory: <pre>\$ cd m-mtca-awg-python/</pre> 5. Verify communication with the hardware by reading its version: <pre>\$ python3 gen_cli.py --info 12:13:54:478 GEN-CLI INFO : Firmware version: 1.0 12:13:54:478 GEN-CLI INFO : Board model: MFMC-DAC 12:13:54:478 GEN-CLI INFO : Device file in use /dev/gen_12 12:13:54:479 GEN-CLI INFO : Board is equipped with ONE FMC</pre> Note down the device file to use, as it will be used in the consecutive tests. In case of having several MTCA-AWG boards in the system a particular one can be selected by specifying the device file name (e.g. /dev/gen_12) as the additional last argument provided to the <code>gen_cli.py</code> script. 6. Download binary file with pulse samples from ITER SharePoint using this link.
Pass Criteria	1. Source code successfully downloaded from GIT. 2. No errors when reading the board info.

TP-MTCA-AWG-F-0002	Test AWG with GUI and Oscilloscope																																																																																
Description	Generate waveforms from Python GUI and observe them with the oscilloscope																																																																																
Pre-requisite	TP-MTCA-AWG-F-0001																																																																																
Procedure	1. Connect the oscilloscope to both outputs of the AWG. 2. Enter the Python scripts directory, if not done so already. 3. Open the AWG control panel: \$ python3 panel.py 4. For each channel: a. Set the signal parameters from the following table. b. Press “Apply” button for that particular channel. c. Using the oscilloscope verify if the expected waveforms are generated. (In order to reduce signal reflections it is recommended to enable 50 Ω termination in the oscilloscope. However, please note that upon doing so the observed voltages will be reduced by half.) <table><tr><th>Test no.</th><th>Shape</th><th>Frequency</th><th>Amplitude</th><th>Offset</th></tr><tr><td>1</td><td>sine</td><td>1 MHz</td><td>1 V</td><td>0 V</td></tr><tr><td>2</td><td>sine</td><td>10 MHz</td><td>1 V</td><td>0 V</td></tr><tr><td>3</td><td>sine</td><td>100 MHz</td><td>1 V</td><td>0 V</td></tr><tr><td>4</td><td>sine</td><td>1 MHz</td><td>1 V</td><td>5 V</td></tr><tr><td>5</td><td>sine</td><td>10 MHz</td><td>1 V</td><td>5 V</td></tr><tr><td>6</td><td>sine</td><td>100 MHz</td><td>1 V</td><td>5 V</td></tr><tr><td>7</td><td>sine</td><td>1 MHz</td><td>1 V</td><td>-5 V</td></tr><tr><td>8</td><td>sine</td><td>10 MHz</td><td>1 V</td><td>-5 V</td></tr><tr><td>9</td><td>sine</td><td>100 MHz</td><td>1 V</td><td>-5 V</td></tr><tr><td>10</td><td>sine</td><td>1 MHz</td><td>5 V</td><td>0 V</td></tr><tr><td>11</td><td>sine</td><td>10 MHz</td><td>5 V</td><td>0 V</td></tr><tr><td>12</td><td>sine</td><td>100 MHz</td><td>5 V</td><td>0 V</td></tr><tr><td>13</td><td>sine</td><td>1 MHz</td><td>5 V</td><td>4.5 V</td></tr><tr><td>14</td><td>sine</td><td>10 MHz</td><td>5 V</td><td>4.5 V</td></tr><tr><td>15</td><td>sine</td><td>100 MHz</td><td>5 V</td><td>4.5 V</td></tr></table>	Test no.	Shape	Frequency	Amplitude	Offset	1	sine	1 MHz	1 V	0 V	2	sine	10 MHz	1 V	0 V	3	sine	100 MHz	1 V	0 V	4	sine	1 MHz	1 V	5 V	5	sine	10 MHz	1 V	5 V	6	sine	100 MHz	1 V	5 V	7	sine	1 MHz	1 V	-5 V	8	sine	10 MHz	1 V	-5 V	9	sine	100 MHz	1 V	-5 V	10	sine	1 MHz	5 V	0 V	11	sine	10 MHz	5 V	0 V	12	sine	100 MHz	5 V	0 V	13	sine	1 MHz	5 V	4.5 V	14	sine	10 MHz	5 V	4.5 V	15	sine	100 MHz	5 V	4.5 V
Test no.	Shape	Frequency	Amplitude	Offset																																																																													
1	sine	1 MHz	1 V	0 V																																																																													
2	sine	10 MHz	1 V	0 V																																																																													
3	sine	100 MHz	1 V	0 V																																																																													
4	sine	1 MHz	1 V	5 V																																																																													
5	sine	10 MHz	1 V	5 V																																																																													
6	sine	100 MHz	1 V	5 V																																																																													
7	sine	1 MHz	1 V	-5 V																																																																													
8	sine	10 MHz	1 V	-5 V																																																																													
9	sine	100 MHz	1 V	-5 V																																																																													
10	sine	1 MHz	5 V	0 V																																																																													
11	sine	10 MHz	5 V	0 V																																																																													
12	sine	100 MHz	5 V	0 V																																																																													
13	sine	1 MHz	5 V	4.5 V																																																																													
14	sine	10 MHz	5 V	4.5 V																																																																													
15	sine	100 MHz	5 V	4.5 V																																																																													

16	sine	1 MHz	5 V	-4.5 V
17	sine	10 MHz	5 V	-4.5 V
18	sine	100 MHz	5 V	-4.5 V
19	sine	1 MHz	9.5 V	0 V
20	sine	10 MHz	9.5 V	0 V
21	triangle	1 MHz	1 V	0 V
22	triangle	10 MHz	1 V	0 V
23	triangle	1 MHz	1 V	5 V
24	triangle	10 MHz	1 V	5 V
25	triangle	1 MHz	1 V	-5 V
26	triangle	10 MHz	1 V	-5 V
27	triangle	1 MHz	5 V	0 V
28	triangle	10 MHz	5 V	0 V
29	triangle	1 MHz	5 V	4.5 V
30	triangle	10 MHz	5 V	4.5 V
31	triangle	1 MHz	5 V	-4.5 V
32	triangle	10 MHz	5 V	-4.5 V
33	triangle	1 MHz	9.5 V	0 V
34	triangle	10 MHz	9.5 V	0 V
35	square	1 MHz	1 V	0 V
36	square	10 MHz	1 V	0 V
37	square	1 MHz	1 V	5 V
38	square	10 MHz	1 V	5 V
39	square	1 MHz	1 V	-5 V
40	square	10 MHz	1 V	-5 V
41	square	1 MHz	5 V	0 V

42	square	10 MHz	5 V	0 V
43	square	1 MHz	5 V	4.5 V
44	square	10 MHz	5 V	4.5 V
45	square	1 MHz	5 V	-4.5 V
46	square	10 MHz	5 V	-4.5 V
47	square	1 MHz	9.5 V	0 V
48	square	10 MHz	9.5 V	0 V
49	bi-exp	1 MHz	1 V	0 V
50	bi-exp	10 MHz	1 V	0 V
51	bi-exp	1 MHz	1 V	5 V
52	bi-exp	10 MHz	1 V	5 V
53	bi-exp	1 MHz	1 V	-5 V
54	bi-exp	10 MHz	1 V	-5 V
55	bi-exp	1 MHz	5 V	0 V
56	bi-exp	10 MHz	5 V	0 V
57	bi-exp	1 MHz	5 V	4.5 V
58	bi-exp	10 MHz	5 V	4.5 V
59	bi-exp	1 MHz	5 V	-4.5 V
60	bi-exp	10 MHz	5 V	-4.5 V
61	bi-exp	1 MHz	9.5 V	0 V
62	bi-exp	10 MHz	9.5 V	0 V

Pass Criteria

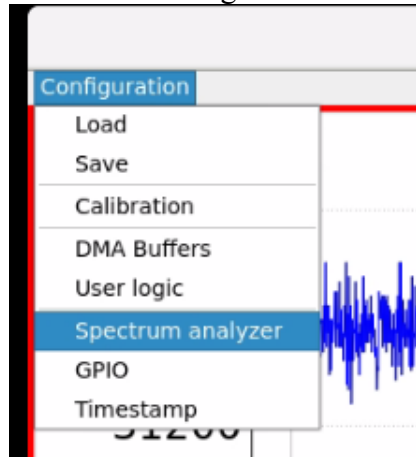
1. All waveforms were generated with expected:
 - a. Frequency – with error less than 5%
 - b. Amplitude – with error less than 5%
 - c. Offset – with error less than 200 mV

TP-MTCA-AWG-F-0003	Test AWG with Oscilloscope and MATLAB data
Description	Generate a waveform using MATLAB and observe it with the oscilloscope
Pre-requisite	TP-MTCA-AWG-F-0001
Procedure	1. Open MATLAB and execute the following script: <pre> # Window definition f_s = 1e9; #Hz end_t = 0.01; #s # Function parameters a_sin = 1; #V f_sin = 100; #Hz (exactly one period in the window - make nice looping possible) # Firmware requires the buffer to contain number of samples divisible by 32 num_samples = f_s * end_t; word_size = 32 num_words = round(num_samples / word_size); # Start file output file = fopen("/tmp/wave.bin", "w"); # Pre-calculate constant parts of the generator function w_sin = 2 * pi * f_sin; a_sin = a_sin / 10 * (2 ^ 15); # volts to DAC counts # Generate 8 samples in each loop revolution for w = 0:num_words-1 first_sample = w * word_size; # Calculate time instants in which to evaluate the generator function t = linspace(first_sample/f_s, (first_sample+word_size-1)/f_s, word_size); # Implement the generator function val = a_sin * sin(w_sin * t); # Store results to file fwrite(file, val, 'int16', 'l'); end fclose(file); </pre> Alternatively, download the generated file from this link. 2. Connect the oscilloscope to channel 1 of the AWG. 3. Enter the Python scripts directory, if not done so already. 4. Initialize the AWG: <pre> \$ python3 gen_cli.py -i </pre> 5. Start the generator: <pre> \$ python3 gen_cli.py --loop /tmp/wave.bin </pre> 6. Use the oscilloscope to examine the resulting waveform.
Pass Criteria	
	1. Sine wave of 100 Hz frequency and 1 V amplitude observed on the oscilloscope.

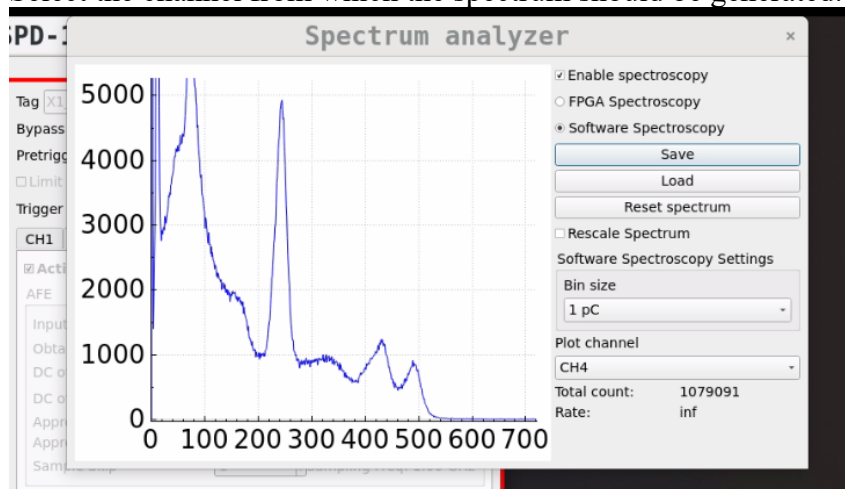
TP-MTCA-AWG-F-0004	Observe Emulated PMT Pulses with Oscilloscope
Description	Setup waveform from Python CLI and observe them with the oscilloscope
Pre-requisite	TP-MTCA-AWG-F-0001
Procedure	1. Connect the oscilloscope to both outputs of the AWG. 2. Enter the Python scripts directory, if not done so already. 3. Initialize the AWG: <pre>\$ python3 gen_cli.py -i</pre> 4. Start the generator: <pre>\$ python3 gen_cli.py --loop ADQ_GEN_DIRECT_CO60.bin</pre> <pre>14:17:00:306 GEN-CLI INFO : Firmware version: 1.0</pre> <pre>14:17:00:306 GEN-CLI INFO : Loading data</pre> <pre>14:17:00:519 GEN-CLI INFO : Starting playback</pre> <pre>14:17:00:519 GEN-CLI INFO : Finished</pre> 5. Using the oscilloscope observe the resulting waveform. (Recommended timescale: 2 μs/div, trigger level: -1 V.) 6. Optionally, initialize the board again to stop the generator: <pre>\$ python3 gen_cli.py -i</pre>
Pass Criteria	1. Waveform similar to this shall be observed:  Points to check: • Baseline around 0 V • Negative pulse every 2 μs

TP-MTCA-AWG-F-0005	Validate AWG operation with ADQ14 (Optional)
Description	Load samples to the AWG, receive them in ADQ14 and compare spectrum to the reference.
Pre-requisite	TP-MTCA-AWG-F-0001
Procedure	1. Connect first output of the AWG to channel 4 of ADQ14 via 20 dB attenuator. 2. Enter the Python scripts directory, if not done so already. 3. Initialize the AWG: <pre>\$ python3 gen_cli.py -i 12:30:45:258 GEN-CLI INFO : Firmware version: 1.0 12:30:45:819 GEN-CLI INFO : FMC1: External PLL is locked 12:30:45:819 GEN-CLI INFO : FMC1: REFCLK = 250.014 MHz 12:30:45:820 GEN-CLI INFO : FMC1: External clock OK 12:30:45:820 GEN-CLI INFO : FMC1: DAQ_CLK = 125.007 MHz</pre> 4. Run the ADQscope application <pre>\$./qtadqscope</pre> 5. Import settings using Configuration → Load A screenshot of the ADQscope application's Configuration menu. The menu is open, showing options: Configuration, Load, Save, Calibration, DMA Buffers, User logic, Spectrum analyzer, GPIO, and Timestamp. The 'Load' option is highlighted with a blue background. A red vertical line is visible on the left side of the menu. 6. Find “adq14_adqgen_settings.json” and load it 7. Disable saving the data to file: A screenshot of the ADQscope Processing window. It shows a 'File save' dropdown menu set to 'DISABLED', which is highlighted with a red rectangle. To the right of the dropdown is a 'SW TRIG' button.

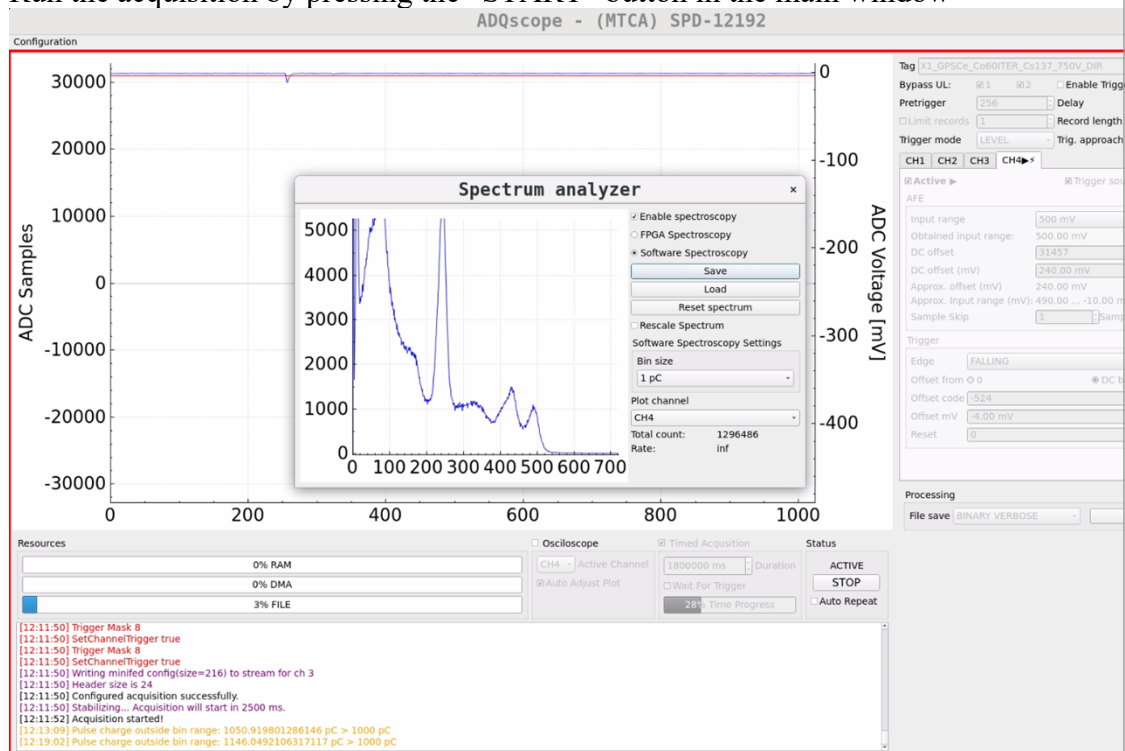
8. From the “Configuration” menu select “Spectrum analyzer”



9. Select the channel from which the spectrum should be generated.



10. Run the acquisition by pressing the “START” button in the main window



11. Start the generator:

```
$ python3 gen_cli.py --once ADQ_GEN_DIRECT_CO60.bin --slow
12:36:56:228    GEN-CLI    INFO : Firmware version: 1.0
12:36:56:229    GEN-CLI    INFO : Loading data
12:37:06:381    GEN-CLI    INFO : Starting playback
12:37:22:943    GEN-CLI    INFO : Finished
```

12. Verify whether the spectrum updates during the operation.
13. After the Python script finishes, check the resulting spectrum and save it to the .csv file using “Save” button in the “Spectrum Analyzer” window.

Pass Criteria

1. The obtained spectrum has 4 distinctive peaks, similar to those shown in point 8.

3.5 MicroTCA.4 FPGA Reference System STR Template

3.5.1 Summary

Test scope	Number of tests	Pass	Pass with message	Skip	Fail	Success ratio
Hardware installation	1					
Software installation	4					
Bitstream synthesis and upload	2					
Functional tests	5					

3.5.2 Hardware Installation Tests

TP-MTCA-AWG-H-0001	Power on MicroTCA.4 chassis and check PCIe operation	[PASS / FAIL]
[Exceptional test results, comments or notes]		

3.5.3 Software Installation Tests

TP-MTCA-AWG-S-0001	Download and install the dmtcauni driver	[PASS / FAIL]
[Exceptional test results, comments or notes]		

TP-MTCA-AWG-S-0002	dmtcauni driver uninstall procedure	[PASS / FAIL]
[Exceptional test results, comments or notes]		

TP-MTCA-AWG-S-0003	Download and install the ADQ14 device drivers	[PASS / FAIL]
[Exceptional test results, comments or notes]		

TP-MTCA-AWG-S-0004	Download and install the ADQscope	[PASS / FAIL]
[Exceptional test results, comments or notes]		

3.5.4 Bitstream Synthesis and Upload Tests

TP-MTCA-AWG-B-0001	Download and build the FPGA project	[PASS / FAIL]
[Exceptional test results, comments or notes]		

TP-MTCA-AWG-B-0002	Program the FPGA PROM	[PASS / FAIL]
[Exceptional test results, comments or notes]		

3.5.5 Functional Tests

TP-MTCA-AWG-F-0001	Download and test the Python Support	[PASS / FAIL]
[Exceptional test results, comments or notes]		

TP-MTCA-AWG-F-0002	Test AWG with GUI and Oscilloscope	[PASS / FAIL]
[Exceptional test results, comments or notes]		

TP-MTCA-AWG-F-0003	Test AWG with Oscilloscope and MATLAB data	[PASS / FAIL]
[Exceptional test results, comments or notes]		

TP-MTCA-AWG-F-0004	Observe Emulated PMT Pulses with Oscilloscope	[PASS / FAIL]
[Exceptional test results, comments or notes]		

TP-MTCA-AWG-F-0005	Validate AWG operation with ADQ14	[PASS / FAIL]
[Exceptional test results, comments or notes]		